

# QA Authoring Studio – Architecture Review & Implementation Plan

## Background

I have been building a Claude Skills package for generating enterprise-grade Test Plans and Test Cases for the NextBillion.ai Route Optimization API.

The current project contains:

- Knowledge Base
- Skills
- Templates
- OpenAPI Specification
- Business Rules
- API Documentation
- Test Plan Generator
- Test Case Generator

The current implementation works by prompting Claude directly.

I now want to evolve this into a complete **local desktop QA Authoring Studio** rather than a prompt collection.

This application will run only on my own machine.

It will **not** be deployed publicly.

It should therefore prioritize:

- simplicity
- maintainability
- speed
- lightweight architecture

over enterprise scalability.

---

## Vision

Instead of manually writing prompts, I want a graphical application that allows me to design a testing scenario.

The application should generate the prompt automatically, send it to Claude, receive structured results, display them in the UI, and allow me to save them for future reuse.

Claude should become the reasoning engine, while the application manages configuration, persistence, and user interaction.

---

## Desired Workflow

### Step 1

User selects:

- Region
  - Number of Vehicles
  - Number of Jobs
  - Test Types
  - Output Type
- 

### Step 2

The application displays the Route Optimization API feature tree.

Example:

Vehicles

- Capacity
- Skills
- Costs
- Breaks
- Time Windows
- Alternative Capacities

Jobs

- Pickup
- Delivery
- Priority
- Amount
- Skills

Shipments

Options

Relations

Zones

Every feature (including nested fields) should be selectable using checkboxes.

These checkboxes should ideally be generated from the OpenAPI schema rather than hardcoded.

---

### Step 3

The user selects only the fields they want included in the generated scenario.

Example:

✓ Capacity

✓ Skills

✓ Priority

✓ Time Windows

✓ Routing

---

### Step 4

The backend builds a structured configuration object.

Example:

```
{
  "region": "USA",
  "vehicles": 5,
  "jobs": 25,
  "features": [
    "vehicles.capacity",
    "vehicles.skills",
    "jobs.priority",
    "jobs.time_windows"
  ],
  "testTypes": [
    "Positive",
    "Negative",
    "API"
  ]
}
```

The frontend never builds prompts.

The backend owns all prompt engineering.

---

## Step 5

The backend builds the Claude prompt automatically.

Claude receives:

- OpenAPI knowledge
- Knowledge Base
- Constraints
- Templates
- User configuration

Claude generates:

- realistic depots
  - realistic vehicles
  - realistic jobs
  - request payload
  - expected responses
  - assertions
  - complete test cases
- 

## Dynamic Test Data Generation

I do NOT want predefined Jobs or Vehicles.

Claude should generate them dynamically according to the selected scenario.

Example:

User chooses:

Region = USA

Vehicles = 5

Jobs = 20

Features:

- Capacity
- Skills
- Priority

Claude creates:

- vehicles
- jobs
- depots

- request payload

using those inputs.

---

## Geographic Data

The only predefined data I want is location data.

For every supported region I plan to maintain around 100 locations.

Example:

USA

locations.json

India

locations.json

Australia

locations.json

Europe

locations.json

Each contains:

- latitude
- longitude
- city
- state

The backend randomly selects locations based on the requested scenario.

Claude uses those coordinates while generating realistic jobs and vehicles.

Therefore:

Locations are predefined.

Business objects are generated dynamically.

---

## Claude Output

I do NOT want Claude to return Markdown as the primary output.

Instead, Claude should return structured JSON.

Example:

```
{
  "testCaseId": "TC001",
  "title": "Verify Vehicle Capacity",
  "category": "Positive",
  "request": {},
  "expectedResponse": {},
  "assertions": []
}
```

The frontend renders this JSON.

Markdown, Excel, PDF, and Word become export formats.

---

## Saving

After generation:

Frontend displays the generated test cases.

The user can:

- edit
- approve
- reject
- save

Once saved, the test cases become reusable.

Future generations can be compared against previously saved suites.

---

## Versioning

I want versioning support.

For every generated suite I want metadata such as:

- Region
- Vehicle Count

- Job Count
- Selected Features
- Prompt Version
- Knowledge Version
- API Version
- Generated Date

This should allow comparing two generated suites in the future.

---

## Suggested Technology

Current thinking:

Frontend

- React
- Vite
- TypeScript
- Tailwind

Backend

- Node.js
- Express or Fastify

Storage

- SQLite

Claude

- Claude API
- 

## Existing Project

The current project already contains:

Knowledge/

Skills/

Templates/

OpenAPI

Business Rules

I want to reuse these rather than rebuild them.

---

# What I Need From You

Please review this architecture critically.

Do NOT simply agree with it.

Instead:

1. Identify architectural weaknesses.
2. Suggest improvements.
3. Recommend a better folder structure if necessary.
4. Suggest how to organize Prompt Builder.
5. Suggest how to organize the Knowledge Layer.
6. Suggest how to organize the Configuration Layer.
7. Suggest how to organize frontend components.
8. Suggest backend APIs.
9. Suggest SQLite schema.
10. Suggest how the feature tree should be generated automatically from OpenAPI.
11. Suggest how prompts should be versioned.
12. Suggest how generated test cases should be stored.
13. Suggest how future modules (Playwright, BugBug, JSONata, Test Plans, Benchmarking) can plug into the same architecture.
14. Suggest an implementation roadmap with milestones.
15. Identify anything that may become difficult to maintain in the future.
16. Recommend best practices for building a local AI-assisted QA Authoring Studio that is extensible and maintainable for years.

Finally, propose an end-to-end architecture diagram and explain the complete flow from UI → Backend → Claude → Storage → Export.